

Mapas a escala

Usar QGIS para generar cientos de mapas

Repositorio Github:

`github.com/paulbradshaw/QGIS_param`

En 1 hora sabrás:

- Cómo usar **Python en QGIS** (funcionalidad Python)
- Cómo usar la **parametrización** para generar un mapa por cada zona/socio/ciudad
- Consejos para usar **IA** para ayudarte a programar

Escenario:

La BBC Shared Data Unit está trabajando en un reportaje sobre defensas contra inundaciones. El reportaje se distribuirá a redacciones de todo el Reino Unido. Queremos generar una imagen de mapa para la redacción de cada ciudad que muestre el estado de las defensas contra inundaciones en su zona.

**¿Cómo creo un
mapa?**

Hazlo ahora:

**Encuentra el archivo
shapefile**

Importar en QGIS

Sigue la guía del repositorio

1. Ve a [AIMS Spatial Flood Defences \(inc. standardised attributes\)](#) y descarga el archivo llamado `AIMS_Spatial_Flood_Defences_inc_standardised_attributes.shp.zip`
2. Descomprime ese archivo
3. Abre QGIS y crea un proyecto nuevo (*Project > New*). Guárdalo en la misma carpeta.
4. Selecciona *Layer > Add Layer > Add vector layer*.
5. En la ventana que aparece, haz clic en ... junto a la sección Fuente (donde dice *Vector Dataset(s)*) y localiza el archivo en la carpeta descargada que termina en **.shp**. Haz clic en **Añadir**. Debería aparecer en el fondo detrás de la ventana. Haz clic en **OK** y luego **Cerrar** para volver a eso.
6. En el *Browser* en la parte superior izquierda de la pantalla, desplázate hacia abajo hasta la sección XYZ Tiles. Haz doble clic en OpenStreetMap para añadir esa capa base a tu proyecto (haz clic en OK para aprobar el cambio al mismo CRS). Ahora debería aparecer en el área de Capas.
7. Haz clic y arrastra la capa OSM para moverla por debajo de la capa de defensas contra inundaciones.

Personalizar la apariencia

1. Haz clic derecho en la capa de defensas contra inundaciones y selecciona **Properties...**
2. Selecciona **Symbology** para cambiar el color, opacidad, grosor, estilo, etc. de las líneas/formas

**¿Cómo exporto
una imagen?**

Hazlo ahora:

Ampliar a una zona
Project > New Print
Layout...

Crear un diseño de impresión (lo automatizaremos)

1. Amplía la parte del mapa de la que quieres crear una imagen (p. ej. con zoom sobre una ciudad)
2. Ve a **Project > New Print Layout...** Dale un nombre (p. ej. cityzoom) y haz clic en **OK**
3. Aparecerá la ventana del diseño, en blanco. Haz clic en el botón Añadir Mapa a la izquierda (icono de página con un signo más verde)
4. Haz clic y arrastra para dibujar el área donde quieres que aparezca tu mapa.
5. Ve a la pestaña **Propiedades del elemento** en la parte derecha y desplázate hacia abajo hasta **Posición y tamaño**. Cámbialo a las dimensiones que quieras para tu imagen (cambia la medida a píxeles si es para web).
6. Cambia a la pestaña **Diseño** siguiente, y desplázate hacia abajo hasta el botón **Cambiar tamaño** del diseño. Haz clic en él. La imagen debería ocupar todo el lienzo (puedes ampliar usando los botones de zoom en la esquina superior izquierda)
7. Cuando estés satisfecho con la imagen, ve a **Layout > Export as Image**
8. Cierra la ventana de diseño de impresión y vuelve al proyecto QGIS



**¿Cómo hago eso
varias veces?**

Si tienes que hacer algo más de una vez...

Plugins > Python Console

Escribe/abre Python en la Consola **debajo del mapa.**

*Haz clic en el botón **Mostrar editor** para abrir/editar archivos .py*

PyQGIS library ([documentación](#))

Hazlo ahora:

Descarga el archivo .py

Ábrelo en tu Consola

Standardise

Python Console

```
1 # Python Console
2 # Use iface to access QGIS API interface or type '?' for more info
3 # Security warning: typing commands from an untrusted source can harm your computer
4
```

>>>



Untitled-0 X

1

Coordinate

357074,195749



Scale

1:152681



Magnifier

100%

Rotation

0.0 °



Render



EPSG:27700



(Pasos en la parte 3 de la guía del repositorio)

1. Ve al [archivo raw](#) en GitHub. Haz clic en Archivo > Guardar como... en tu navegador y guarda el archivo en tu carpeta de proyecto
2. Descomenta esta línea y cambia la ruta para que apunte a tu carpeta de proyecto

```
# out_prefix = "/Users/paul/Downloads/testqgis/images/"
```

(Para conocer la ruta a tu carpeta en Mac, haz clic derecho en ella y selecciona Obtener información. La ventana que aparece te mostrará la ruta en 'Dónde:')
3. En la Consola Python de QGIS, abre el archivo y ejecuta el código

```
# Cargar herramientas que nos permiten controlar mapas y diseños de QGIS desde este script
```

```
from qgis.PyQt.QtCore import QRectF
```

```
from qgis.core import (
```

```
    QgsProject, QgsCoordinateReferenceSystem, QgsPrintLayout, QgsLayoutItemMap,
```

```
    QgsLayoutPoint, QgsLayoutSize, QgsUnitTypes, QgsLayoutItemLabel,
```

```
    QgsLayoutExporter, QgsRectangle, QgsCoordinateTransform, QgsPointXY
```

```
)
```

```
# Cargar math para números, re (expresiones regulares) para texto, os para navegación de archivos
```

```
import math, re
```

```
import os
```

Aquí se define la carpeta de salida

Usar la librería os para crear una carpeta en tu ordenador donde guardar las imágenes de mapa exportadas.

Se colocará en tu carpeta de Descargas, dentro de una subcarpeta llamada "qgis_images".

Si la carpeta ya existe, no hace nada – no sobrescribirá nada.

```
out_dir = os.path.join(os.path.expanduser("~"), "Downloads", "qgis_images")
```

```
os.makedirs(out_dir, exist_ok=True)
```

```
out_prefix = out_dir + "/"
```

Para sustituir la carpeta configurada arriba, descomenta y adapta la línea siguiente

```
# out_prefix = "/Users/paul/Downloads/testqgis/images/"
```

#establecer el Sistema de Referencia de Coordenadas (CRS) - EPSG:4326 se usa para GPS
#pero 27700 se usa para las defensas contra inundaciones del RU, así que lo usamos para mayor rigor

```
QgsProject.instance().setCrs(QgsCoordinateReferenceSystem("EPSG:27700"))
```

Obtener el proyecto QGIS actualmente abierto y guardarlo en una variable llamada project

```
project = QgsProject.instance()
```

#Usar EPSG:27700 significa que debemos convertir lat/long a coordenadas BNG
primero establecer los sistemas de origen (EPSG:4326) y destino (EPSG:27700)

```
source_crs = QgsCoordinateReferenceSystem("EPSG:4326")
```

```
dest_crs = QgsCoordinateReferenceSystem("EPSG:27700")
```

#luego crear un transformador que use esos dos

```
transformer = QgsCoordinateTransform(  
    source_crs,  
    dest_crs,  
    project  
)
```

#crear un nuevo diseño de impresión en ese proyecto, guardar en una variable llamada layout

layout = QgsPrintLayout(project)

#Rellenarlo con la configuración predeterminada

layout.initializeDefaults()

#Asignarle un nombre en QGIS

layout.setName("centres_layout") # Give the layout a name

Añadir un panel de mapa al diseño – este es el rectángulo que mostrará el mapa.

```
map_item = QgsLayoutItemMap(layout)
```

```
map_item.setFrameEnabled(True) # Dibujar un borde alrededor del panel de mapa
```

Posicionar el panel de mapa a 20mm desde la izquierda y 20mm desde la parte superior de la página.

```
map_item.attemptMove(QgsLayoutPoint(20, 20, QgsUnitTypes.LayoutMillimeters))
```

Establecer el panel de mapa a 257mm de ancho y 170mm de alto (aproximadamente A4 apaisado menos márgenes).

```
map_item.attemptResize(QgsLayoutSize(257, 170, QgsUnitTypes.LayoutMillimeters))
```

```
layout.addLayoutItem(map_item) # Añadir el panel de mapa a la página del diseño
```

Añadir una etiqueta de texto al diseño, que mostrará el nombre y las coordenadas de cada ubicación.

```
label = QgsLayoutItemLabel(layout)
```

```
layout.addLayoutItem(label)
```

Posicionar la etiqueta cerca de la parte superior de la página (20mm desde la izquierda, 5mm desde arriba).

```
label.attemptMove(QgsLayoutPoint(20, 5, QgsUnitTypes.LayoutMillimeters))
```

def crea una función (una receta) que podemos usar más tarde. Esta función calcula un área de mapa rectangular centrada en un punto dado. Necesita cuatro ingredientes: x e y del punto central; y la mitad del ancho/alto (con un valor por defecto para el último si no se proporciona).

```
def rect_around(x, y, half_width, half_height=None):
```

```
    # Si no se proporciona altura, usar el mismo valor que el ancho
```

```
    if half_height is None:
```

```
        half_height = half_width
```

```
    # Cuando se use, devolverá un rectángulo que se puede usar para definir qué muestra el mapa.
```

```
    return QgsRectangle(
```

```
        x - half_width,
```

```
        y - half_height,
```

```
        x + half_width,
```

```
        y + half_height )
```

Aquí es donde se almacenan las ubicaciones

Crear una lista de ubicaciones para mapear. Cada elemento tiene una latitud, longitud y un nombre.

Empezamos con solo dos elementos para probar, pero una vez que funcione, lo ampliaremos a todas las ubicaciones para las que queremos crear imágenes

```
listofdicstocsv = [  
    {'lat': 50.844441271809465, 'long': -0.2985307978506628, 'officialname': 'Adur  
District Council'},  
    {'lat': 54.71108952940033, 'long': -3.2472347297148736, 'officialname':  
'Allerdale Borough Council'}  
]
```

Aquí es donde se define la **escala**

ESTABLECER ESCALA DEL MAPA: 1 unidad en el mapa equivale a 175000 en realidad

MAP_SCALE = 175000

Aquí es donde se define la salida

```
# Configurar la herramienta que guardará el diseño como archivo de imagen.  
exporter = QgsLayoutExporter(layout)  
  
settings = QgsLayoutExporter.ImageExportSettings()  
  
# ESTABLECER TAMAÑO DE IMAGEN DE SALIDA  
  
# 1200x795px para uso web, coincidiendo con la proporción del diseño A4 del  
# código anterior  
  
settings.imageSize = QSize(1200, 795)
```

```
# Recorrer cada ubicación de la lista, y usar la función para crear/exportar una imagen

for rec in listofdicstocsv:

    # Leer la latitud, longitud y nombre de esta ubicación.

    lat = rec.get('lat', float('nan')) # 'nan' se usa como sustituto si el valor falta
    lon = rec.get('long', float('nan'))

    officialname = rec.get('officialname', 'unknown')

    # Si falta la latitud o la longitud, omitir esta ubicación y continuar.

    if math.isnan(lat) or math.isnan(lon):

        print(f"skipping '{officialname}': missing lat/long")

        continue

    # Mover la vista del mapa para que quede centrada en esta ubicación, usando lon y lat

    transformed_point = transformer.transform( QgsPointXY(lon, lat) )

    # continúa...
```

```
# Extraer las coordenadas transformadas

x = transformed_point.x()
y = transformed_point.y()

#Crear un rectángulo temporal alrededor del punto para centrarlo
map_item.setExtent( rect_around(x, y, 1000, 1000) )

#Establecer una escala para controlar el zoom. Cada mapa usará esa.
map_item.setScale(MAP_SCALE)

#añadir una etiqueta con esa información
label.setText( f"{officialname} | scale 1:{MAP_SCALE:,.}" )

#actualizar el mapa antes de exportar
map_item.refresh()

# continúa...
```

```
#dar un nombre de archivo - eliminar caracteres problemáticos

safe_name = re.sub( r'^A-Za-z0-9._-]+', '_', officialname).strip('_')

#usar eso para determinar la ruta donde guardar

out_path = ( f"{out_prefix}" f"{safe_name}.png" )

#y exportar a esa ruta

result = exporter.exportToImage( out_path, settings )

#si no fue exitoso

if result != QgsLayoutExporter.Success:

    #mostrar un mensaje de error

    raise RuntimeError(f"Export failed for {out_path}")

#imprimir un mensaje al final de cada iteración

print("wrote", out_path)
```

**¿Cómo hago eso
258 veces?**

Repaso: dónde se almacenan las ubicaciones

Crear una lista de ubicaciones para mapear. Cada elemento tiene una latitud, longitud y un nombre.

Empezamos con solo dos elementos para probar, pero una vez que funcione, lo ampliaremos a todas las ubicaciones para las que queremos crear imágenes

```
listofdicstocsv = [  
    {'lat': 50.844441271809465, 'long': -0.2985307978506628, 'officialname': 'Adur  
District Council'},  
    {'lat': 54.71108952940033, 'long': -3.2472347297148736, 'officialname':  
'Allerdale Borough Council'}  
]
```

Convertir un CSV a una lista de diccionarios

1. **Descargar el CSV de ciudades** (haz clic en los tres puntos en la esquina superior derecha)
2. **Escribe esta fórmula en la celda J2:**
`=("{'lat': "&B2&", 'long': "&C2&", 'officialname': '"&A2&"'}")"`
3. **Copia la fórmula hacia abajo para cada fila. Ahora tienes una columna de diccionarios.**
4. **Usa esta fórmula para unir los resultados y poner una coma entre cada uno**
`=TEXTJOIN(", ", TRUE, J:J)`
5. **En tu código Python, encuentra los [corchetes] de apertura y cierre** que contienen tu lista de dos elementos, y reemplaza esos dos elementos con la nueva lista de diccionarios, conservando los corchetes.

{lat: 51.5072, 'long': -0.1275, 'officialname': 'London'}, {lat: 52.48, 'long': -1.9025, 'officialname': 'Birmingham'}, {lat: 50.8058, 'long': -1.0872, 'officialname': 'Portsmouth'}, {lat: 50.9025, 'long': -1.4042, 'officialname': 'Southampton'}, {lat: 52.9561, 'long': -1.1512, 'officialname': 'Nottingham'}, {lat: 51.4536, 'long': -2.5975, 'officialname': 'Bristol'}, {lat: 53.479, 'long': -2.2452, 'officialname': 'Manchester'}, {lat: 53.4094, 'long': -2.9785, 'officialname': 'Liverpool'}, {lat: 52.6344, 'long': -1.1319, 'officialname': 'Leicester'}, {lat: 50.8147, 'long': -0.3714, 'officialname': 'Worthing'}, {lat: 52.4081, 'long': -1.5106, 'officialname': 'Coventry'}, {lat: 54.5964, 'long': -5.93, 'officialname': 'Belfast'}, {lat: 53.8, 'long': -1.75, 'officialname': 'Bradford'}, {lat: 52.9247, 'long': -1.478, 'officialname': 'Derby'}, {lat: 50.3714, 'long': -4.1422, 'officialname': 'Plymouth'}, {lat: 51.4947, 'long': -0.1353, 'officialname': 'Westminster'}, {lat: 52.5833, 'long': -2.1333, 'officialname': 'Wolverhampton'}, {lat: 52.2304, 'long': -0.8938, 'officialname': 'Northampton'}, {lat: 52.6286, 'long': 1.2928, 'officialname': 'Norwich'}, {lat: 51.8783, 'long': -0.4147, 'officialname': 'Luton'}, {lat: 52.413, 'long': -1.778, 'officialname': 'Solihull'}, {lat: 51.544, 'long': -0.1027, 'officialname': 'Islington'}, {lat: 57.15, 'long': -2.11, 'officialname': 'Aberdeen'}, {lat: 51.3727, 'long': -0.1099, 'officialname': 'Croydon'}, {lat: 50.72, 'long': -1.88, 'officialname': 'Bournemouth'}, {lat: 51.58, 'long': 0.49, 'officialname': 'Basildon'}, {lat: 51.272, 'long': -0.529, 'officialname': 'Maidstone'}, {lat: 51.5575, 'long': 0.0858, 'officialname': 'Ilford'}, {lat: 53.39, 'long': -2.59, 'officialname': 'Warrington'}, {lat: 51.75, 'long': -1.25, 'officialname': 'Oxford'}, {lat: 51.5836, 'long': -0.3464, 'officialname': 'Harrow'}, {lat: 52.519, 'long': -1.995, 'officialname': 'West Bromwich'}, {lat: 51.8667, 'long': -2.25, 'officialname': 'Gloucester'}, {lat: 53.96, 'long': -1.08, 'officialname': 'York'}, {lat: 53.8142, 'long': -3.0503, 'officialname': 'Blackpool'}, {lat: 53.4083, 'long': -2.1494, 'officialname': 'Stockport'}, {lat: 53.424, 'long': -2.322, 'officialname': 'Sale'}, {lat: 51.5975, 'long': -0.0681, 'officialname': 'Tottenham'}, {lat: 52.2053, 'long': 0.1192, 'officialname': 'Cambridge'}, {lat: 51.5768, 'long': 0.1801, 'officialname': 'Romford'}, {lat: 51.8917, 'long': 0.903, 'officialname': 'Colchester'}, {lat: 51.6287, 'long': -0.7482, 'officialname': 'High Wycombe'}, {lat: 54.9556, 'long': -1.6, 'officialname': 'Gateshead'}, {lat: 51.5084, 'long': 0.1192, 'officialname': 'Slough'}, {lat: 53.748, 'long': -2.482, 'officialname': 'Blackburn'}, {lat: 51.73, 'long': 0.48, 'officialname': 'Chelmsford'}, {lat: 53.61, 'long': -2.16, 'officialname': 'Rochdale'}, {lat: 53.43, 'long': -1.357, 'officialname': 'Rotherham'}, {lat: 51.584, 'long': -0.021, 'officialname': 'Walthamstow'}, {lat: 51.2667, 'long': -1.0876, 'officialname': 'Basingstoke'}, {lat: 53.483, 'long': -2.2931, 'officialname': 'Salford'}, {lat: 51.4668, 'long': -0.375, 'officialname': 'Hounslow'}, {lat: 51.5528, 'long': -0.2979, 'officialname': 'Wembley'}, {lat: 52.1911, 'long': -2.2206, 'officialname': 'Worcester'}, {lat: 51.4928, 'long': -0.2229, 'officialname': 'Hammersmith'}, {lat: 51.5864, 'long': 0.6049, 'officialname': 'Rayleigh'}, {lat: 51.7526, 'long': -0.4692, 'officialname': 'Hemel Hempstead'}, {lat: 51.38, 'long': -2.36, 'officialname': 'Bath'}, {lat: 51.5127, 'long': -0.4211, 'officialname': 'Hayes'}, {lat: 54.527, 'long': -1.5526, 'officialname': 'Darlington'}, {lat: 50.8352, 'long': -0.1758, 'officialname': 'Hove'}, {lat: 50.85, 'long': 0.57, 'officialname': 'Hastings'}, {lat: 51.655, 'long': -0.3957, 'officialname': 'Watford'}, {lat: 51.9017, 'long': -0.2019, 'officialname': 'Stevenage'}, {lat: 54.69, 'long': -1.21, 'officialname': 'Hartlepool'}, {lat: 53.19, 'long': -2.89, 'officialname': 'Chester'}, {lat: 51.4828, 'long': -0.195, 'officialname': 'Fulham'}, {lat: 52.523, 'long': -1.468, 'officialname': 'Nuneaton'}, {lat: 51.5175, 'long': -0.2988, 'officialname': 'Ealing'}, {lat: 51.8168, 'long': -0.8124, 'officialname': 'Aylesbury'}, {lat: 51.6154, 'long': -0.0708, 'officialname': 'Edmonton'}, {lat: 51.755, 'long': -0.336, 'officialname': 'Saint Albans'}, {lat: 53.789, 'long': -2.248, 'officialname': 'Burnley'}, {lat: 53.7167, 'long': -1.6356, 'officialname': 'Batley'}, {lat: 53.5809, 'long': -0.6502, 'officialname': 'Scunthorpe'}, {lat: 52.508, 'long': -2.089, 'officialname': 'Dudley'}, {lat: 51.4575, 'long': -0.1175, 'officialname': 'Brixton'}, {lat: 51.5111, 'long': -0.3756, 'officialname': 'Southall'}, {lat: 55.8456, 'long': -4.4239, 'officialname': 'Paisley'}, {lat: 51.37, 'long': 0.52, 'officialname': 'Chatham'}, {lat: 53.523, 'long': 0.0554, 'officialname': 'East Ham'}, {lat: 51.346, 'long': -2.977, 'officialname': 'Weston-super-Mare'}, {lat: 54.8947, 'long': -2.9364, 'officialname': 'Carlisle'}, {lat: 54.995, 'long': -1.43, 'officialname': 'South Shields'}, {lat: 55.7644, 'long': -4.1769, 'officialname': 'East Kilbride'}, {lat: 52.8019, 'long': -1.6367, 'officialname': 'Burton upon Trent'}, {lat: 53.9919, 'long': -1.5378, 'officialname': 'Harrogate'}, {lat: 53.099, 'long': -2.44, 'officialname': 'Crewe'}, {lat: 52.48, 'long': 1.75, 'officialname': 'Lewes'}, {lat: 52.37, 'long': -1.26, 'officialname': 'Rugby'}, {lat: 51.623, 'long': 0.009, 'officialname': 'Chingford'}, {lat: 51.5404, 'long': -0.4778, 'officialname': 'Uxbridge'}, {lat: 52.58, 'long': -1.98, 'officialname': 'Walsall'}, {lat: 51.475, 'long': 0.33, 'officialname': 'Grays'}, {lat: 51.3868, 'long': -0.4133, 'officialname': 'Walton upon Thames'}, {lat: 51.4002, 'long': -0.1086, 'officialname': 'Thornthorpe Heath'}, {lat: 51.599, 'long': -0.187, 'officialname': 'Finchley'}, {lat: 51.5, 'long': -0.19, 'officialname': 'Kensington'}, {lat: 52.974, 'long': -0.0214, 'officialname': 'Boston'}, {lat: 50.4353, 'long': -3.5625, 'officialname': 'Paignton'}, {lat: 50.88, 'long': -1.03, 'officialname': 'Waterlooville'}, {lat: 53.875, 'long': -1.706, 'officialname': 'Guisely'}, {lat: 51.5565, 'long': 0.2128, 'officialname': 'Hornchurch'}, {lat: 51.4009, 'long': -0.1517, 'officialname': 'Mitcham'}, {lat: 51.4496, 'long': -0.4089, 'officialname': 'Feltham'}, {lat: 52.4575, 'long': -2.1479, 'officialname': 'Stourbridge'}, {lat: 51.375, 'long': 0.5, 'officialname': 'Rochester'}, {lat: 53.691, 'long': -1.633, 'officialname': 'Dewsbury'}, {lat: 51.5135, 'long': -0.2707, 'officialname': 'Acton'}, {lat: 51.449, 'long': -0.337, 'officialname': 'Twickenham'}, {lat: 53.046, 'long': -2.993, 'officialname': 'Wrexham'}, {lat: 53.279, 'long': -2.897, 'officialname': 'Elesmere Port'}, {lat: 54.66, 'long': -5.67, 'officialname': 'Bangor'}, {lat: 51.019, 'long': -3.1, 'officialname': 'Taunton'}, {lat: 52.7225, 'long': -1.2078, 'officialname': 'Loughborough'}, {lat: 51.54, 'long': 0.08, 'officialname': 'Barking'}, {lat: 51.6185, 'long': -0.2729, 'officialname': 'Edgware'}, {lat: 50.8094, 'long': -0.5409, 'officialname': 'Littlehampton'}, {lat: 51.576, 'long': -0.433, 'officialname': 'Ruislip'}, {lat: 51.4279, 'long': -0.1235, 'officialname': 'Streatham'}, {lat: 51.132, 'long': 0.263, 'officialname': 'Royal Tunbridge Wells'}, {lat: 53.35, 'long': -3.003, 'officialname': 'Bebington'}, {lat: 53.25, 'long': -2.13, 'officialname': 'Macclesfield'}, {lat: 52.3028, 'long': -0.6944, 'officialname': 'Wellingborough'}, {lat: 52.3931, 'long': -0.7229, 'officialname': 'Kettering'}, {lat: 51.878, 'long': 0.55, 'officialname': 'Brintree'}, {lat: 52.2919, 'long': -1.5358, 'officialname': 'Royal Leamington Spa'}, {lat: 54.1108, 'long': -3.2261, 'officialname': 'Barrow in Furness'}, {lat: 56.0719, 'long': -3.4393, 'officialname': 'Dunfermline'}, {lat: 53.3838, 'long': -2.3547, 'officialname': 'Altrincham'}, {lat: 54.0489, 'long': -2.8014, 'officialname': 'Lancaster'}, {lat: 53.4872, 'long': -3.0343, 'officialname': 'Crosby'}, {lat: 53.4457, 'long': -2.9891, 'officialname': 'Bootle'}, {lat: 51.5423, 'long': -0.0026, 'officialname': 'Stratford'}, {lat: 51.0792, 'long': 1.1794, 'officialname': 'Fekstone'}, {lat: 55.945, 'long': -3.994, 'officialname': 'Cumbemauld'}, {lat: 51.208, 'long': -1.48, 'officialname': 'Andover'}, {lat: 51.66, 'long': -3.81, 'officialname': 'Neath'}, {lat: 52.488, 'long': -2.05, 'officialname': 'Rowley Regis'}, {lat: 54.2825, 'long': -0.4, 'officialname': 'Scarborough'}, {lat: 55.98, 'long': -3.17, 'officialname': 'Leith'}, {lat: 50.9452, 'long': -2.637, 'officialname': 'Yeovil'}, {lat: 51.451, 'long': 0.052, 'officialname': 'Eltham'}, {lat: 51.5541, 'long': -0.1744, 'officialname': 'Hampstead'}, {lat: 53.125, 'long': -1.261, 'officialname': 'Sutton in Ashfield'}, {lat: 51.4015, 'long': -0.1949, 'officialname': 'Morden'}, {lat: 51.6444, 'long': -0.1997, 'officialname': 'Barnet'}, {lat: 53.4466, 'long': -2.3086, 'officialname': 'Stretford'}, {lat: 51.408, 'long': -0.022, 'officialname': 'Beckenham'}, {lat: 51.5299, 'long': -0.3488, 'officialname': 'Greenford'}, {lat: 51.702, 'long': -0.035, 'officialname': 'Cheshunt'}, {lat: 53.48, 'long': -2.89, 'officialname': 'Kirkby'}, {lat: 51.07, 'long': -1.79, 'officialname': 'Salsbury'}, {lat: 53.4897, 'long': -2.0952, 'officialname': 'Ashton'}, {lat: 51.394, 'long': -0.307, 'officialname': 'Sutton'}, {lat: 53.716, 'long': -1.356, 'officialname': 'Castleford'}, {lat: 51.4452, 'long': -0.0207, 'officialname': 'Catford'}, {lat: 53.3042, 'long': -1.1244, 'officialname': 'Workshop'}, {lat: 53.7492, 'long': -1.6023, 'officialname': 'Morley'}, {lat: 51.743, 'long': -3.378, 'officialname': 'Merthyr Tydfil'}, {lat: 53.555, 'long': -2.187, 'officialname': 'Middleton'}, {lat: 51.2834, 'long': -0.8456, 'officialname': 'Fleet'}, {lat: 50.85, 'long': -1.18, 'officialname': 'Fareham'}, {lat: 53.4487, 'long': -2.3747, 'officialname': 'Urmston'}, {lat: 51.3656, 'long': -0.1963, 'officialname': 'Sutton'}, {lat: 51.578, 'long': -3.218, 'officialname': 'Caerphilly'}, {lat: 51.128, 'long': -2.993, 'officialname': 'Bridgwater'}, {lat: 51.401, 'long': -1.323, 'officialname': 'Newbury'}, {lat: 51.4594, 'long': 0.1097, 'officialname': 'Welling'}, {lat: 51.46, 'long': -2.505, 'officialname': 'Kingswood'}, {lat: 51.886, 'long': -0.521, 'officialname': 'Dunstable'}, {lat: 51.336, 'long': 1.416, 'officialname': 'Ramsgate'}, {lat: 51.393, 'long': 0.478, 'officialname': 'Strood'}, {lat: 53.5533, 'long': -0.0215, 'officialname': 'Cleethorpes'}, {lat: 51.5932, 'long': -0.3894, 'officialname': 'Pinner'}, {lat: 52.606, 'long': 1.729, 'officialname': 'Great Yarmouth'}, {lat: 52.9711, 'long': -1.3092, 'officialname': 'Ilkeston'}, {lat: 53.653, 'long': -2.632, 'officialname': 'Chorley'}, {lat: 51.37, 'long': 1.13, 'officialname': 'Heme Bay'}, {lat: 51.872, 'long': 0.1725, 'officialname': 'Bishops Cleeve'}, {lat: 53.005, 'long': -1.127, 'officialname': 'Arnold'}, {lat: 52.724, 'long': -1.369, 'officialname': 'Coalville'}, {lat: 51.994, 'long': -0.732, 'officialname': 'Bletchley'}, {lat: 51.9165, 'long': -0.6617, 'officialname': 'Leighton Buzzard'}, {lat: 55.86, 'long': -3.98, 'officialname': 'Ardrie'}, {lat: 55.126, 'long': -1.514, 'officialname': 'Blyth'}, {lat: 51.574, 'long': 0.4181, 'officialname': 'Laindon'}, {lat: 51.684, 'long': -4.163, 'officialname': 'Llanelli'}, {lat: 52.927, 'long': -1.215, 'officialname': 'Beeston'}, {lat: 52.4629, 'long': -1.8542, 'officialname': 'South Heath'}, {lat: 55.0456, 'long': -1.4443, 'officialname': 'Whitley Bay'}, {lat: 53.4554, 'long': -2.112, 'officialname': 'Denton'}, {lat: 52.932, 'long': -1.127, 'officialname': 'West Bridgford'}, {lat: 51.6578, 'long': -0.2722, 'officialname': 'Borehamwood'}, {lat: 56.0011, 'long': -3.7835, 'officialname': 'Falkirk'}, {lat: 53.5239, 'long': -2.3991, 'officialname': 'Walden'}, {lat: 51.5878, 'long': -0.3086, 'officialname': 'Kenton'}, {lat: 54.0819, 'long': -0.1923, 'officialname': 'Bridlington'}, {lat: 54.61, 'long': -1.27, 'officialname': 'Billingham'}, {lat: 52.918, 'long': -0.638, 'officialname': 'Grantham'}, {lat: 55.0097, 'long': -1.4448, 'officialname': 'North Shields'}, {lat: 51.947, 'long': -0.283, 'officialname': 'Hitchin'}, {lat: 52.7858, 'long': -0.1529, 'officialname': 'Spalding'}, {lat: 51.36, 'long': 0.61, 'officialname': 'Rainham'}, {lat: 51.978, 'long': -0.23, 'officialname': 'Letchworth'}, {lat: 51.6114, 'long': 0.5207, 'officialname': 'Wickford'}, {lat: 53.4111, 'long': -2.8403, 'officialname': 'Huyton'}, {lat: 51.6717, 'long': -1.2763, 'officialname': 'Abingdon'}, {lat: 51.32, 'long': -2.208, 'officialname': 'Trowbridge'}, {lat: 52.5812, 'long': -1.093, 'officialname': 'Wigston Magna'}, {lat: 51.606, 'long': -1.241, 'officialname': 'Didcot'}, {lat: 51.433, 'long': -0.933, 'officialname': 'Eaerley'}, {lat: 51.459, 'long': 0.138, 'officialname': 'Bexleyheath'}, {lat: 53.4429, 'long': -1.4698, 'officialname': 'Ecclesfield'}, {lat: 53.698, 'long': -2.461, 'officialname': 'Darwen'}, {lat: 53.5333, 'long': -2.2833, 'officialname': 'Prestwich'}, {lat: 51.602, 'long': -3.342, 'officialname': 'Pontypridd'}, {lat: 55.828, 'long': -4.214, 'officialname': 'Rutherglen'}, {lat: 51.1295, 'long': 1.3089, 'officialname': 'Dover'}, {lat: 50.8365, 'long': -0.7792, 'officialname': 'Chichester'}, {lat: 51.2226, 'long': 1.4006, 'officialname': 'Deal'}, {lat: 51.9, 'long': -1.15, 'officialname': 'Bicester'}, {lat: 51.5467, 'long': -0.37, 'officialname': 'Northolt'}, {lat: 55.7742, 'long': -3.9183, 'officialname': 'Wishaw'}, {lat: 51.3652, 'long': -0.1676, 'officialname': 'Carshalton'}, {lat: 53.001, 'long': -1.197, 'officialname': 'Bulwell'}, {lat: 54.591, 'long': -5.68, 'officialname': 'Newtownards'}, {lat: 54.326, 'long': -2.745, 'officialname': 'Kendal'}, {lat: 55.082, 'long': -1.585, 'officialname': 'Cramlington'}, {lat: 52.3353, 'long': -2.0579, 'officialname': 'Bromsgrove'}, {lat: 51.703, 'long': -3.041, 'officialname': 'Pont-y-pŵll'}, {lat: 51.509, 'long': -0.338, 'officialname': 'Hanwell'}, {lat: 51.2279, 'long': -2.3215, 'officialname': 'Frome'}, {lat: 51.5981, 'long': -0.1149, 'officialname': 'Wood Green'}, {lat: 52.5708, 'long': -2.0457, 'officialname': 'Darlaston'}, {lat: 55.181, 'long': -1.568, 'officialname': 'Ashington'}, {lat: 52.9677, 'long': -2.1327, 'officialname': 'Longton'}, {lat: 52.7661, 'long': -0.886, 'officialname': 'Melton Mowbray'}, {lat: 52.606, 'long': -1.9179, 'officialname': 'Aldridge'}, {lat: 53.5452, 'long': -2.3999, 'officialname': 'Farnworth'}, {lat: 51.552, 'long': -0.097, 'officialname': 'Highbury'}, {lat: 53.3761, 'long': -2.1897, 'officialname': 'Chaddle Hulme'}, {lat: 54.62, 'long': -1.58, 'officialname': 'Newton Aycliffe'}, {lat: 52.4299, 'long': -1.9355, 'officialname': 'Bournville'}, {lat: 52.009, 'long': -0.789, 'officialname': 'Shenley Brook End'}, {lat: 54.85, 'long': -1.83, 'officialname': 'Consett'}, {lat: 51.3211, 'long': -0.1386, 'officialname': 'Coulston'}, {lat: 52.566, 'long': -2.0728, 'officialname': 'Bilston'}, {lat: 52.7001, 'long': -2.5157, 'officialname': 'Wellington'}, {lat: 54.663, 'long': -1.676, 'officialname': 'Bishop Auckland'}, {lat: 52.395, 'long': -1.979, 'officialname': 'Longbridge'}, {lat: 52.614, 'long': -2.004, 'officialname': 'Bloxwich'}, {lat: 51.5557, 'long': 0.2512, 'officialname': 'Uxminster'}, {lat: 53.321, 'long': -3.48, 'officialname': 'Rhyll'}, {lat: 52.267, 'long': -2.153, 'officialname': 'Droitwich'}, {lat: 53.5355, 'long': -2.5658, 'officialname': 'Hindley'}, {lat: 53.549, 'long': -2.529, 'officialname': 'Westhoughton'}, {lat: 51.3589, 'long': 1.4394, 'officialname': 'Broadstairs'}

Usa un slice en el bucle for para probarlo:

Añadir [10:20] después del nombre de la lista significa que solo iterará por los elementos en las posiciones 10 a 19

```
for rec in listofdicstocsv[10:20]:
```

```
    # Leer la latitud, longitud y nombre de esta ubicación.
```

```
    lat = float(rec.get('lat', float('nan'))) # 'nan' se usa como sustituto si el valor falta
```

```
    lon = float(rec.get('long', float('nan')))
```

```
    officialname = rec.get('officialname', 'unknown')
```

```
    # Si falta la latitud o la longitud, omitir esta ubicación y continuar.
```

```
    if math.isnan(lat) or math.isnan(lon):
```

```
        print(f"skipping '{officialname}': missing lat/long")
```

```
        continue
```

**¿Cómo hago eso
para áreas de
diferente tamaño?**

**Necesitamos escalas
diferentes:**

**Gran conurbación, área
más pequeña**

Vibe coding ¿es hora del vibe coding?

Usa el meta prompting para pedir a la IA que diseñe el prompt primero

Incluye:

- El contexto (periodismo)
- El objetivo
- **Instrucciones detalladas** (nombres específicos, archivos, etc.)

Eres mi mentor, un periodista de datos con más de una década de experiencia en el campo. Tienes **conocimientos estadísticos avanzados** así como un sano **escepticismo** al tratar tanto con datos como con fuentes humanas. Te pediré ayuda con algo de programación - estás dispuesto a guiarme, pero **no quieres que pierda habilidades ni que dependa demasiado** de ti, por lo que tus consejos siempre estarán diseñados para **obligarme a pensar por mí mismo, aprender nuevas habilidades y conceptos, y practicarlos**.

Cuando proporciones código **añade comentarios que expliquen cada paso** como si fuera para una persona sin experiencia en programación. Yo **prefiero código sencillo que requiera varias líneas antes que código complejo en una o dos líneas**.

Señala cualquier suposición que estés haciendo sobre los datos o mi pregunta. Avísame si la pregunta no contiene suficiente información para responder con precisión, o si el resultado podría ser engañoso sin contexto adicional.

Adjunto hay un Python que genera imágenes para mapas centrados en cientos de ubicaciones. **Adáptalo para que ahora genere esas imágenes a dos escalas diferentes.**

Comenta el código para resaltar la sección donde hace esto, de modo que pueda ajustarlo y probar diferentes niveles de zoom.

Parámetros

[https://github.com/paulbradshaw/QGIS_param/blob/main/qgis_take_pix_t
woSize.py](https://github.com/paulbradshaw/QGIS_param/blob/main/qgis_take_pix_t
woSize.py)

La sección que define los parámetros de escala

```
# ESTABLECER ESCALAS DEL MAPA:
```

```
MAP_SCALES = [  
    175000,    # escala original  
    120000    # versión ampliada  
]
```

La sección que usa esos parámetros

```
# NUEVO: RECORRER MÚLTIPLES ESCALAS

for MAP_SCALE in MAP_SCALES:

    #Crear un rectángulo temporal alrededor del punto para centrarlo
    map_item.setExtent(
        rect_around(x, y, 1000, 1000)
    )

    #Aplicar la escala actual
    map_item.setScale(MAP_SCALE)

    #añadir una etiqueta con esa información
    label.setText(f"{officialname} | scale 1:{MAP_SCALE:,}")
```

...y añadiéndolo al nombre del archivo

#usar eso para determinar la ruta donde guardar

out_path = (

 f"{out_prefix}"

 f"{safe_name}_{MAP_SCALE}.png" # NUEVO: AÑADIR ESCALA AL NOMBRE DEL

ARCHIVO

)

El amigo crítico:
Lo que la IA debería
decirte

Aspectos a tener en cuenta

- **Conversión entre el CRS del mapa base y el shapefile**
- **Conversión entre el tamaño del diseño (mm) e imageSize (px)**
- **Exportar para impresión con resolución de 200 ppp**
- **Categorías de escala en lugar de números brutos (y más de dos)**

Prueba esto:
Otros archivos
opcionales en el
repositorio

Conclusiones clave

- Si tienes que hacerlo más de una vez...
- El **vibe coding** es divertido pero el *mentor coding* es más enriquecedor

Recursos adicionales:

- https://github.com/paulbradshaw/QGIS_param
- Tim Green: [El Vibe Coding amenaza el periodismo: por qué las redacciones necesitan gobernanza ahora](#)
- [FT newsletter](#) sobre su metodología usando IA para asistir con la programación en la cartografía de zonas de riesgo de inundación